CS23532 - COMPUTER NETWORKS LABORATORY

EXPERIMENT 7

AIM:

To write a program to implement flow control at the data link layer using the Sliding Window Protocol and simulate the flow of data frames between two network nodes.

DESCRIPTION:

- **Flow Control at Data Link Layer:** A technique used at the data link layer to regulate the rate of data transmission between two nodes. It prevents a high-speed sender from overwhelming a slower receiver, ensuring proper buffering, frame processing, and zero data loss.
- **Sliding Window Protocol:** Allows the sender to transmit multiple frames before receiving an acknowledgment (ACK). The sender maintains a bounded "sliding window" of frame sequence numbers allowed to be sent, while the receiver maintains a corresponding window of sequence numbers it is prepared to accept.
- **Piggybacking & Bidirectional Flow:** An optimization technique where acknowledgment (ACK) control information is embedded directly into outgoing data frames sent from the receiver back to the sender, maximizing network bandwidth efficiency.

PROGRAM ARCHITECTURE OVERVIEW:

- **Sender Program:** Responsible for taking the message input, dividing it into discrete frames with sequence numbers, maintaining the sliding window boundaries, and writing frames to the shared file buffer ("Sender_Buffer"). It continuously reads receiver responses

("Receiver_Buffer") to adjust the sending window or handle retransmissions.

- **Receiver Program:** Monitors "Sender_Buffer" for incoming frames, validates sequence numbers against expected window indices, and writes acknowledgment (ACK) or negative acknowledgment (NACK) frames back into "Receiver_Buffer".

PROGRAMS AND LOGIC:

Sender Program (sender.py)

Python

```
import time
import os

def sender():
    window_size = int(input("Enter window size: "))
    message = input("Enter the message to send: ")
    frames = [f"Frame {i}: {message[i]}" for i in range(len(message))]

    base = 0
    next_seq_num = 0
    total_frames = len(frames)

    # Clear buffer files
    open("Sender_Buffer.txt", "w").close()
    open("Receiver_Buffer.txt", "w").close()

    while base < total_frames:
        # Send frames within the window
        while next_seq_num < base + window_size and next_seq_num <
total_frames:
            print(f"Sending {frames[next_seq_num]}")
            with open("Sender_Buffer.txt", "a") as f:
```

```
                f.write(f"{frames[next_seq_num]}\n")
            next_seq_num += 1
            time.sleep(1)

        # Check for feedback/ACKs from Receiver
        if os.path.exists("Receiver_Buffer.txt"):
            with open("Receiver_Buffer.txt", "r") as f:
                lines = f.readlines()
            if lines:
                last_ack = lines[-1].strip()
                if "ACK" in last_ack:
                    ack_num = int(last_ack.split()[1])
                    print(f"Received ACK for Frame {ack_num}")
                    base = ack_num + 1
                elif "NACK" in last_ack:
                    nack_num = int(last_ack.split()[1])
                    print(f"Received NACK for Frame {nack_num}.
Retransmitting...")
                    next_seq_num = nack_num
        time.sleep(2)

if __name__ == "__main__":
    sender()
```

Receiver Program (receiver.py)

Python

```
import time
import os

def receiver():
    expected_frame = 0
    processed_lines = 0
```

```
    print("Receiver running... Waiting for frames.")
    while True:
        if os.path.exists("Sender_Buffer.txt"):
            with open("Sender_Buffer.txt", "r") as f:
                lines = f.readlines()

            while processed_lines < len(lines):
                frame_data = lines[processed_lines].strip()
                if frame_data:
                    frame_num = int(frame_data.split(":")[0].split()[1])
                    if frame_num == expected_frame:
                        print(f"Accepted {frame_data}")
                        with open("Receiver_Buffer.txt", "a") as rf:
                            rf.write(f"ACK {frame_num}\n")
                        expected_frame += 1
                    else:
                        print(f"Out of order frame received! Expected
Frame {expected_frame}")
                        with open("Receiver_Buffer.txt", "a") as rf:
                            rf.write(f"NACK {expected_frame}\n")
                processed_lines += 1
        time.sleep(1)

if __name__ == "__main__":
    receiver()
```

STEP-BY-STEP PROCEDURE:

Task 1: Implement and Execute the Sender Program

Prompt the user to enter the transmission window size and text message input.
Construct frames formatted with sequence headers [Frame No., DATA].

Transmit initial frames within the window limit and write states to "Sender_Buffer".
Read feedback status from "Receiver_Buffer" to advance the sliding window on ACK or retransmit frames on NACK.

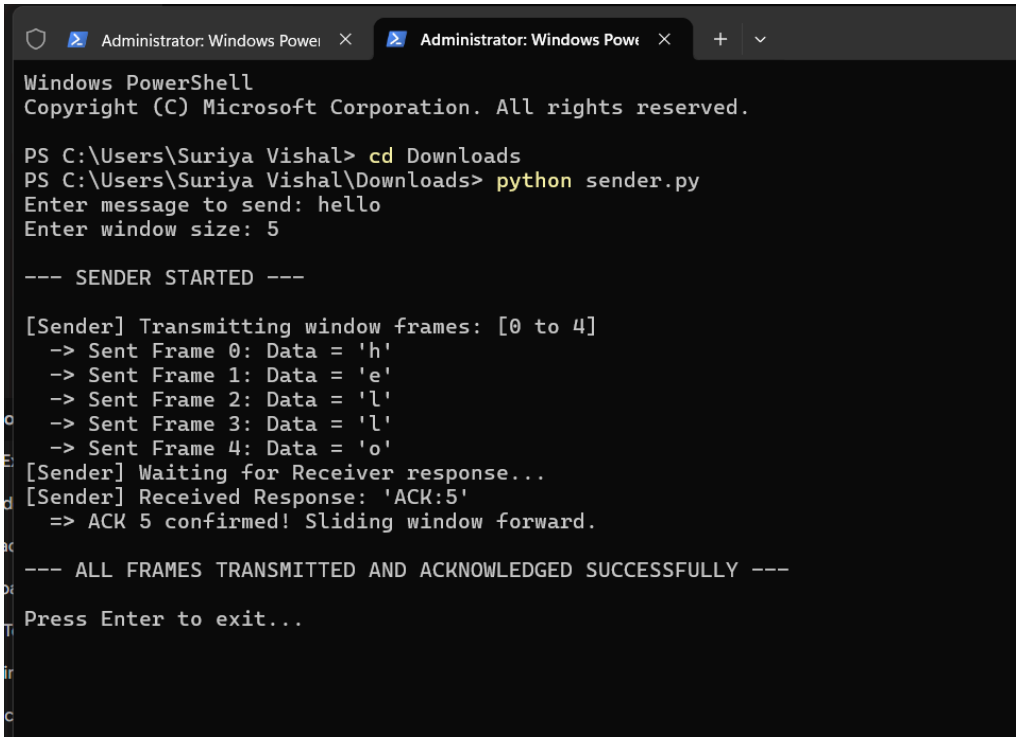
Task 2: Implement and Execute the Receiver Program

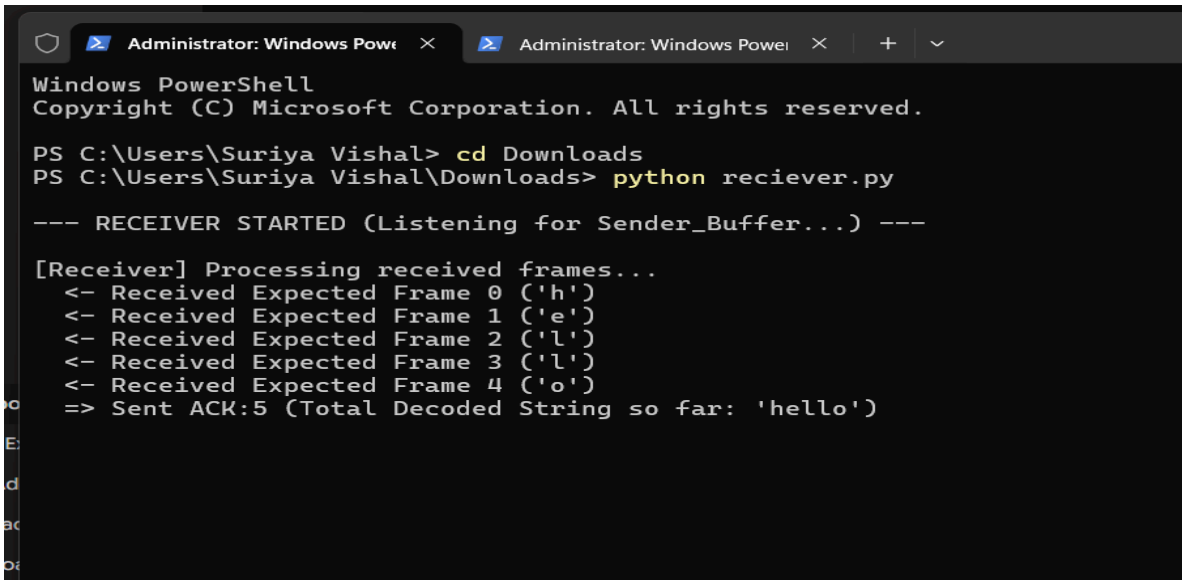
Read transmitted frames directly from "Sender_Buffer".
Inspect sequence numbers against expected indices and generate appropriate ACK or NACK responses into "Receiver_Buffer".

Task 3: Simulate Error Induction and Verify Retransmission

Manually modify frame sequence numbers or ACK flags within "Sender_Buffer" or "Receiver_Buffer".

Execute receiver and sender routines to confirm detection of out-of-order sequence frames and verify automatic retransmission.





Result:

Thus the program to implement flow control at the data link layer using the Sliding Window Protocol and simulate the flow of data frames between two network nodes has been executed successfully.